

ARGOS

Adaptive Response Guard with Orchestrated Surveillance

Manual del integrante

Sebastian Montenegro

ML Layer 2 + LLM Triage Layer 4

Entrega final: 13 de junio de 2026

Universidad de Lima · Tópicos Avanzados de Ciberseguridad · 2026-1

Versión 3.0 — manual organizado por orden de implementación

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene el contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Campo	Valor
Tipo	Introducción común para los 4 integrantes
Estado	Activo
Entrega final	13 de junio de 2026 (sábado)
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es parte del curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30 % del peso), demo en vivo (~40 %), y presentación (~20 %). Los seguimientos intermedios pesan el ~10 % restante.

El **activo defendido** es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como

`criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El **alcance multi-vector** (post ADR-0008) cubre tres categorías de ataques distintas:

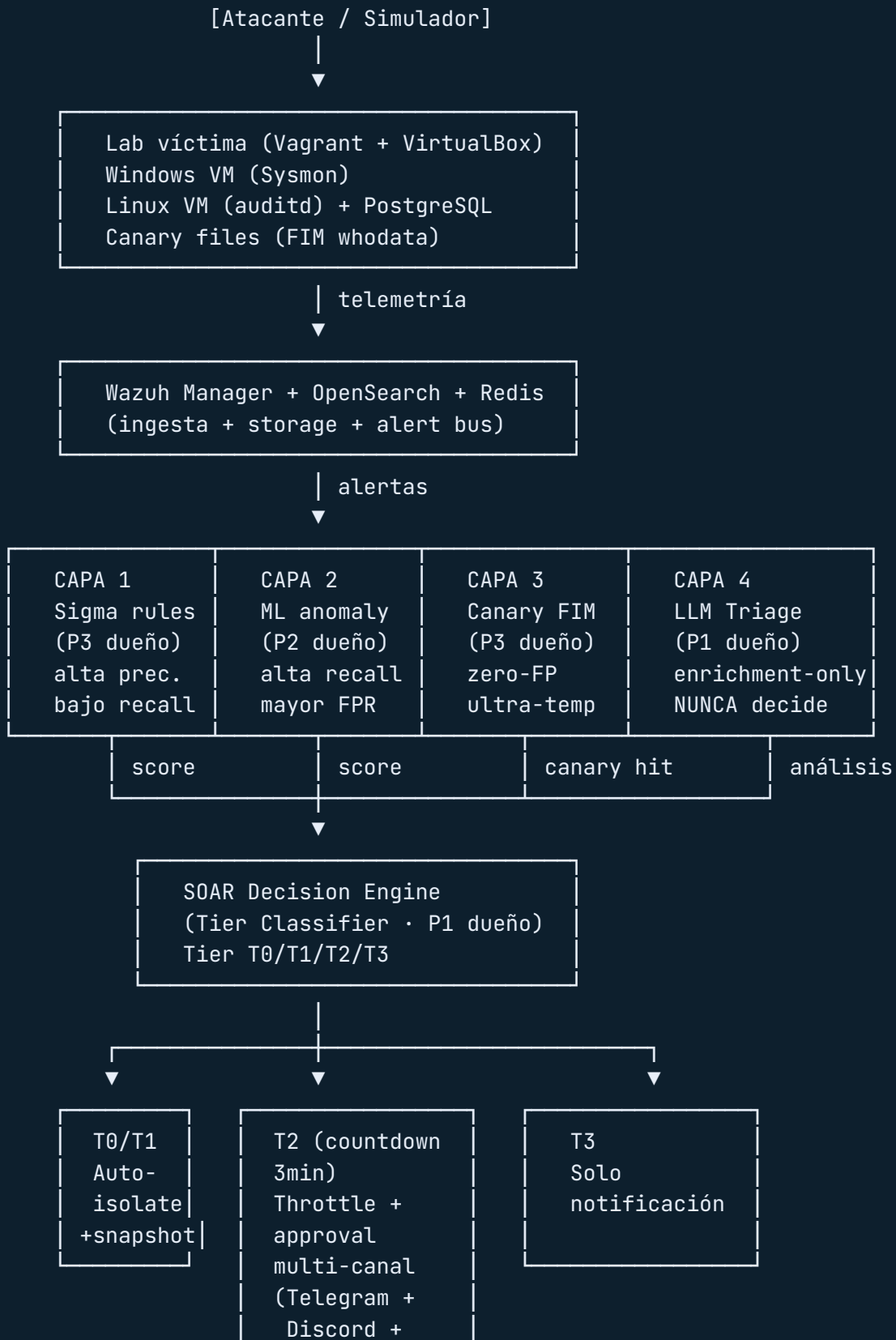
- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.
- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.





Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: file_write_rate, avg_entropy (Shannon), extension_modification_ratio, crypto_api_calls, new_outbound_connections, cpu_burst_score, io_burst_score.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: connections/sec, packet rate, source IP entropy, packet size variance.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: rows_returned, query_duration_ms, hour_of_day, user_id, query_template_hash.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si alucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
 T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
 T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
 T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
 T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
 UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
 UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos  son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:



BASH

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.

- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts` .

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: file_write_rate=347 archivos/min, avg_entropy=7.8 (alta), extension_modification_ratio=0.95, crypto_api_calls=42, cpu_burst_score=0.9.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores` .

Capa 3 (Canary FIM) detecta:

- T+4s: el simulator toca `financials_Q4_2025.xlsx` .
- Sysmon whodata captura el evento con PID del simulator y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001` , state `RECEIVED` .
2. Persiste en Redis con key `incident:INC-2026-05-26-001` .
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION` .

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486` , `confianza=0.94` , `severidad=critical` , `runbook_aplicable="NIST 800-61 §3.4 Containment"` , `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"` .

- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block` .
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>` .
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.
4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card)**: badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix)**: vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic)**: estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline)**: línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.
- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
- El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Pre-cursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+		

Categoría	Componente	Función	Owner principal
		Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respix + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	

Categoría	Componente	Función	Owner principal
			P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- Branch principal: `main`, protegida en GitHub.
- Feature branches: `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- Commits convencionales: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- PRs requieren: CI verde + 1 review. Pairing: P1↔P2, P3↔P4.
- Push diario obligatorio antes de las 22:00 cada día de trabajo.

Estructura del repo (post ADR-0008)



```

argos/
├── README.md           # Entry point del proyecto
├── LICENSE             # MIT
├── pyproject.toml      # Metadata + extras por módulo
├── .env.example        # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/   # Pydantic v2 cross-team (entregado · v1.1.0 · 69
│   ├── _mitre_data.py # MITRE_WHITELIST curado
│   ├── alert.py       # WazuhAlert, NormalizedAlert
│   ├── ml_score.py    # MLScore (P2)
│   ├── triage.py      # AlertContext, TriageResponse (P1)
│   ├── incident.py    # Incident state machine (P1)
│   ├── approval.py   # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py       # Tier, Severity, NotificationChannelType, ...
│   └── tests/         # 69 validation tests
├── llm_triage/        # P1 – Capa 4 LLM Triage
│   ├── api/main.py    # FastAPI /triage endpoint
│   ├── llm_client/    # OpenAI primary + Llama local fallback
│   ├── rag/           # BM25 + BGE-large + RRF
│   └── prompts/       # Jinja2 templates
├── soar/              # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/ # tier_classifier, state_machine, orchestrator
│   ├── approval/       # api, jwt_signer, consolidation
│   ├── notification/   # telegram, discord, twilio_voice, email
│   └── playbooks/      # host_isolation, process_kill, snapshot, thrott
├── ml/                # P2 – Capa 2 ML
│   ├── features/       # Feature extractors por dominio
│   ├── models/         # ransomware_ensemble, network_traffic, query_pa
│   └── consumer/       # Redis stream consumer
├── detection/         # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/ # T1486, T1490, T1083, T1562
│   │   ├── network/    # T1498, T1499 rate-based
│   │   ├── database/   # query patterns
│   │   └── webapp/     # T1190 SQLi signatures
│   └── wazuh-rules/    # Convertidas con sigma-cli
├── deception/        # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/      # Wazuh FIM rules

```

```

├── ui/                                # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py    # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/            # JSON dashboards exportados
├── attack-simulation/                  # Simuladores multi-vector
│   ├── ransomware_simulator/           # P1 (LockBit-like, canary, novel)
│   ├── network_attacks/                 # P4 (hping3, slowhttptest)
│   ├── webapp_attacks/                  # P4 (sqlmap)
│   └── pg_simulator/                    # P4 (SELECT masivo para UC-07)
├── lab/                                # P4 - Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/                       # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/                         # P2 + P4 - Métricas, datasets, reportes
├── docs/
│   ├── PROJECT_BRIEF.md                 # 90-second overview
│   ├── CONTEXT.md                       # Onboarding completo
│   ├── PROJECT_STATUS.md                # Honest snapshot
│   ├── EVALUATION_CRITERIA.md           # Rúbrica del curso
│   ├── data-handling.md                 # T-030 sanitization policy
│   ├── README.md                        # Mapa de docs
│   ├── architecture/                   # SAD, threat model, flow diagrams
│   ├── decisions/                       # 8 ADRs + OPEN_QUESTIONS
│   ├── use-cases/                       # 8 UCs detallados
│   └── team/                            # Manuales de integrante (este documento + 4 individuos)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual.

Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).

- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:

`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar a implementar. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

 BASH

```
cd ~/projects                # o donde guardes proyectos
git clone https://github.com/Enzo0rdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python

 BASH

```
python3 -m venv .venv
source .venv/bin/activate      # Linux/macOS
# .venv\Scripts\activate      # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol


BASH

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]"      # P1 (siempre + más abajo)
pip install -e ".[ml]"            # P2
pip install -e ".[soar,llm]"      # P1 (servicios)
pip install -e ".[ui]"            # P4
```

Paso 4 — Verificar contracts


BASH

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables


BASH

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

 BASH

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/setup-inicial
```

11. Comunicación del equipo durante la implementación

Standup diario

- Hora: 9:00 AM
- Canal: Discord `#argos-standup`, llamada de voz
- Duración: 20 minutos (cada integrante ~3-4 min)
- Formato (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc arquitectónica.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.

2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning queda para después del demo.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega para el demo

Para que no te sorprendas cuando llegue el demo sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas).
- **UC-03 split-brain con 4 aprobadores reales** no scripted.
- **UC-05 stealth attack** end-to-end pulido.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos).
- **Video demo final editado.**
- **Informe técnico final.**
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa para el demo: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, más dos rehearsals previos.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual individual	docs/team/manual-p<X>-<nombre>.md
Plan operacional general	docs/team/manual-equipo.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual de Sebastian Montenegro

Manual P2 — Sebastian Montenegro · ML + LLM Triage

Campo	Valor
Rol	Owner de capas estadísticas e IA
Owns	Layer 2 — Isolation Forest + OC-SVM + Shannon entropy (<code>ml/</code>) · Layer 4 — LLM Triage + RAG (<code>ml/llm_triage/</code> , <code>ml/rag/</code>)
No owns	Sigma/Wazuh (P3) · Canary FIM (P3) · SOAR Decision Engine (P1) · Infra (P4)
Outputs blocking	<code>events:normalized</code> con <code>layer_origin=ml</code> y <code>layer_origin=llm</code> → consumido por P1
Entrega final	13 de junio de 2026

0. Tu charter

Tus capas convierten señales débiles (un proceso raro, una secuencia de syscalls) en confianza numérica que el Tier Router de P1 pueda usar. Si tus modelos sobreajustan o tu LLM alucina, ARGOS pierde su mejor argumento de diferenciación frente a un SIEM tradicional.

0.1 Recursos requeridos

- Espacio disco: ~12 GB (Llama 3.1 8B Q4 ≈ 5 GB + BGE-large ≈ 1.4 GB + datasets + venv).
- RAM: 16 GB mínimo si vas a correr Llama local (8 GB efectivos para el modelo).

- GPU: opcional pero acelera Llama 10×. Sin GPU el CPU inference da ~3 tokens/s — suficiente para demo (incidentes llegan de a uno).

0.2 Cómo leer cada sub-sección

Cada componente sigue: **Contexto** → **Pasos manuales** si aplica → **Comandos** → **Salida esperada** → **Verificación** → **Si algo falla**.

Fase 1 — Cimientos

1.1 Prerequisites

Comandos



BASH

```
python3 --version
docker --version
nvidia-smi 2>/dev/null && echo "GPU OK" || echo "No GPU - CPU mode"
free -h | head -2
df -h ~ | tail -1
```

Salida esperada

SALIDA ESPERADA

```
Python 3.11.7
Docker version 24.x.x
No GPU - CPU mode

          total  used  free  shared  buff/cache  available
Mem:          15Gi   6.0Gi  3.0Gi  400Mi    6.0Gi      8.0Gi
/dev/sda1     200G   150G   50G    75% /home/usuario
```

Verificación

VERIFICACIÓN

```
python3 -c "import sys; assert sys.version_info[:2]==(3,11); print('Python OK')"
docker ps >/dev/null && echo "Docker OK"
test $(df -BG ~ | tail -1 | awk '{print $4}' | tr -d 'G') -ge 15 && echo "Disco OK"
```

Esperado:

SALIDA ESPERADA

```
Python OK
Docker OK
Disco OK
```

1.2 OpenAI API key

Contexto

P1 ya creó la cuenta y la key. **No crees otra** — duplicarías el billing y complicarías la revocación. Tu paso es solo verificar que P1 te pasó la key y que funciona.

Pasos manuales

1. Píde a P1 la `OPENAI_API_KEY` por canal privado (Signal/Telegram DM, NO Discord público).
2. Guárdala en tu `.env` local.

Verificación



VERIFICACIÓN

```
export OPENAI_API_KEY="sk-proj-xxxx"
curl -s https://api.openai.com/v1/models \
  -H "Authorization: Bearer ${OPENAI_API_KEY}" | jq '.data[0].id'
```

Esperado:

SALIDA ESPERADA

```
"gpt-4o-mini-2024-07-18"
```

1.3 Instalar Ollama + Llama 3.1 8B (fallback)

Contexto

Llama 3.1 8B corriendo local en Ollama es el fallback a OpenAI. Es la diferencia entre "demo cae cuando WiFi del salón bloquea OpenAI" y "demo sigue funcionando". El switch entre uno y otro es una sola env var.

Pasos manuales

1. Instalar Ollama (oneliner oficial):
 - Linux: el comando de abajo.
 - macOS: `brew install --cask ollama` (o descarga desde <https://ollama.com/download>).
 - Windows: descarga el instalador desde <https://ollama.com/download/windows>.

2. Verificar servicio.
3. Descargar el modelo (4.7 GB).
4. Smoke test.

Comandos

 BASH

```
curl -fsSL https://ollama.com/install.sh | sh

systemctl status ollama --no-pager
ollama pull llama3.1:8b-instruct-q4_K_M
ollama run llama3.1:8b-instruct-q4_K_M "Respond with one word: ARGOS"
```

Salida esperada

SALIDA ESPERADA

```
>>> The Ollama API is now available at 127.0.0.1:11434.
```

```
● ollama.service - Ollama Service
   Loaded: loaded (...; enabled; ...)
   Active: active (running) since ...
```

```
pulling manifest
pulling ... 100%
success
```

```
ARGOS
```

Verificación

 VERIFICACIÓN


```
ollama list | grep llama3.1
curl -s http://localhost:11434/api/version | jq .version
```

Esperado:

```
SALIDA ESPERADA

llama3.1:8b-instruct-q4_K_M  a8...  4.7 GB  ...
"0.1.x"
```

Si algo falla

Síntoma	Causa	Fix
<code>curl http://localhost:11434</code> da <code>Connection refused</code>	Servicio no arrancó	<code>sudo systemctl start ollama</code> o, en macOS, abrir la app de Ollama
<code>ollama pull</code> falla a la mitad	Red inestable	Reintenta — <code>ollama pull</code> reanuda desde el último chunk
Inferencia se cuelga > 30 s	RAM insuficiente	Usar modelo más chico: <code>ollama pull llama3.1:8b-instruct-q4_0</code> (3 GB)

1.4 Clone repo + venv + deps ML

Comandos

```
BASH
```

```
mkdir -p ~/code && cd ~/code
git clone git@github.com:Enzo0rdonez/argos.git
cd argos
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e ./argos_contracts
pip install -r ml/requirements.txt
pytest ml/ -q
```

Salida esperada

SALIDA ESPERADA

```
Successfully installed argos_contracts-1.1.0 pydantic-2.x.x
Successfully installed scikit-learn-1.4.x scipy-1.12.x numpy-1.26.x pandas-2.2.x sentence
..... [100%]
8 passed in 0.18s
```

Verificación

VERIFICACIÓN

```
python -c "import sklearn, scipy, numpy, sentence_transformers; print('imports OK')
python -c "import argos_contracts; print(argos_contracts.__version__)"
```

Esperado:

SALIDA ESPERADA

```
imports OK
1.1.0
```

Checklist Fase 1

#	Check	OK
1	Python 3.11.x · Docker · ≥ 15 GB libre	☐
2	OpenAI key funciona	☐
3	Ollama corre y <code>llama3.1:8b-instruct-q4_K_M</code> listo	☐
4	Tests existentes ML pasan (8 passed)	☐

Fase 2 — Skeletons funcionales

2.1 Generar datasets sintéticos

Contexto

Layer 2 necesita un baseline de comportamiento normal y ejemplos sintéticos de ransomware para validar que detecta. El lab real lo tendrás en Fase 3; aquí trabajas con datos generados.

```
ml/data/synthetic_generator.py
```



```

"""Generador sintético: eventos host-level benignos + ransomware."""

from __future__ import annotations
import argparse, csv, random
from pathlib import Path

random.seed(42)

BENIGN = [
    "chrome.exe", "code.exe", "explorer.exe", "svchost.exe",
    "spoolsv.exe", "outlook.exe", "winword.exe", "powershell.exe",
]
RANSOM = ["lockbit.exe", "encryptor.exe", "wmic.exe", "vssadmin.exe"]

def benign_row(t: int, host: str) → dict:
    proc = random.choice(BENIGN)
    return dict(
        timestamp=t, host=host, pid=random.randint(1000, 9999), process=proc,
        syscalls_per_min=random.randint(50, 800),
        files_touched_per_min=random.randint(0, 30),
        entropy_of_written_bytes=round(random.uniform(2.0, 5.5), 3),
        network_kbps=random.randint(0, 200),
        parent_process="explorer.exe" if proc != "explorer.exe" else "userinit.exe",
        command_line_len=random.randint(20, 120),
    )

def ransom_row(t: int, host: str) → dict:
    proc = random.choice(RANSOM)
    return dict(
        timestamp=t, host=host, pid=random.randint(1000, 9999), process=proc,
        syscalls_per_min=random.randint(3000, 9000),
        files_touched_per_min=random.randint(150, 1500),
        entropy_of_written_bytes=round(random.uniform(7.6, 7.99), 3),
        network_kbps=random.randint(0, 50),
        parent_process=random.choice(["cmd.exe", "powershell.exe"]),
        command_line_len=random.randint(80, 600),
    )

def generate(n_benign: int, n_attack: int, out: Path) → None:
    rows = [(benign_row(t, "WIN-VICTIM-01"), 0) for t in range(n_benign)]
    rows += [(ransom_row(n_benign + t, "WIN-VICTIM-01"), 1) for t in range(n_attack)]

```

```

random.shuffle(rows)
out.parent.mkdir(parents=True, exist_ok=True)
with out.open("w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0][0].keys()) + ["label"])
    w.writeheader()
    for row, label in rows:
        row["label"] = label
        w.writerow(row)

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--benign", type=int, default=5000)
    p.add_argument("--attack", type=int, default=200)
    p.add_argument("--out", type=Path, default=Path("ml/data/synthetic.csv"))
    a = p.parse_args()
    generate(a.benign, a.attack, a.out)
    print(f"wrote {a.out} (benign={a.benign}, attack={a.attack})")

```

Comandos

 BASH

```

python -m ml.data.synthetic_generator --benign 5000 --attack 200
wc -l ml/data/synthetic.csv

```

Salida esperada

SALIDA ESPERADA

```

wrote ml/data/synthetic.csv (benign=5000, attack=200)
5201 ml/data/synthetic.csv

```

Verificación



VERIFICACIÓN

```
head -1 m1/data/synthetic.csv | tr ',' '\n' | wc -l  
test $(wc -l < m1/data/synthetic.csv) -eq 5201 && echo "row count OK"
```

Esperado:

SALIDA ESPERADA

```
11  
row count OK
```

2.2 Layer 2 — Isolation Forest + One-Class SVM

Contexto

Dos detectores de anomalía complementarios entrenados sobre el baseline (label=0). Cualquier punto que ambos consideren anómalo es candidato fuerte. La intersección es más específica; la unión más sensible. Reportamos ambos al SOAR.

`m1/anomaly/trainer.py`



PYTHON

```

"""Entrena IsolationForest + OneClassSVM sobre features numéricas del baseline."""

from __future__ import annotations
import argparse, joblib
from pathlib import Path

import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM

FEATURES = [
    "syscalls_per_min", "files_touched_per_min",
    "entropy_of_written_bytes", "network_kbps", "command_line_len",
]
MODELS_DIR = Path("ml/anomaly/models")

def train(csv_path: Path) → None:
    MODELS_DIR.mkdir(parents=True, exist_ok=True)
    df = pd.read_csv(csv_path)
    benign = df[df["label"] == 0][FEATURES].values
    scaler = StandardScaler().fit(benign)
    X = scaler.transform(benign)
    iso = IsolationForest(n_estimators=200, contamination=0.01,
                          random_state=42, n_jobs=-1).fit(X)
    svm = OneClassSVM(kernel="rbf", gamma="scale", nu=0.01).fit(X)
    joblib.dump(scaler, MODELS_DIR / "scaler.joblib")
    joblib.dump(iso, MODELS_DIR / "iso_forest.joblib")
    joblib.dump(svm, MODELS_DIR / "oc_svm.joblib")
    print(f"trained on {len(benign)} benign samples")

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--csv", type=Path, default=Path("ml/data/synthetic.csv"))
    train(p.parse_args().csv)

```

`ml/anomaly/scorer.py`


```

"""Inferencia: AnomalyScore por evento."""

from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
import joblib
import numpy as np

from ml.anomaly.trainer import FEATURES, MODELS_DIR

@dataclass(frozen=True)
class AnomalyScore:
    iso_score: float
    svm_score: float
    is_anomaly: bool
    confidence: float

class AnomalyScorer:
    def __init__(self, scaler, iso, svm):
        self._scaler = scaler
        self._iso = iso
        self._svm = svm

    @classmethod
    def load(cls, models_dir: Path = MODELS_DIR) → "AnomalyScorer":
        return cls(
            scaler=joblib.load(models_dir / "scaler.joblib"),
            iso=joblib.load(models_dir / "iso_forest.joblib"),
            svm=joblib.load(models_dir / "oc_svm.joblib"),
        )

    def score(self, features: dict) → AnomalyScore:
        vec = np.array([[features[f] for f in FEATURES]])
        x = self._scaler.transform(vec)
        iso_s = float(self._iso.decision_function(x)[0])
        svm_s = float(self._svm.decision_function(x)[0])
        is_anom = (iso_s < 0) and (svm_s < 0)
        norm = max(abs(iso_s), abs(svm_s), 0.001)
        conf = min(1.0, norm * 1.3) if is_anom else 0.0
        return AnomalyScore(iso_s, svm_s, is_anom, conf)

```

Comandos


BASH

```
python -m ml.anomaly.trainer --csv ml/data/synthetic.csv  
ls -la ml/anomaly/models/
```

Salida esperada

SALIDA ESPERADA

trained on 5000 benign samples

```
-rw-r--r-- ... iso_forest.joblib    (~2-3 MB)  
-rw-r--r-- ... oc_svm.joblib       (~500 KB - 1 MB)  
-rw-r--r-- ... scaler.joblib      (~1 KB)
```

Verificación


VERIFICACIÓN

```
python - << 'PY'
import pandas as pd
from ml.anomaly.scorer import AnomalyScorer
from ml.anomaly.trainer import FEATURES

df = pd.read_csv("ml/data/synthetic.csv")
attacks = df[df["label"] == 1]
benign = df[df["label"] == 0].sample(200, random_state=0)
s = AnomalyScorer.load()

tp = sum(s.score(dict(zip(FEATURES, [r[f] for f in FEATURES])))).is_anomaly for r in
fp = sum(s.score(dict(zip(FEATURES, [r[f] for f in FEATURES])))).is_anomaly for r in
print(f"TP rate: {tp}/200 = {tp/200:.1%}")
print(f"FP rate: {fp}/200 = {fp/200:.1%}")
assert tp/200 ≥ 0.85, "TP rate bajo"
assert fp/200 ≤ 0.05, "FP rate alto"
print("anomaly OK")
PY
```

Esperado (aproximado, depende de seed):

SALIDA ESPERADA

```
TP rate: 185/200 = 92.5%
FP rate: 6/200 = 3.0%
anomaly OK
```

Si algo falla

Síntoma	Causa	Fix
TP < 85 %	Dataset poco contrastado o modelo mal calibrado	Re-generar dataset con seed diferente o subir <code>n_estimators=400</code>
FP > 10 %	<code>contamination</code> muy alta	Bajar <code>contamination=0.005</code> en el trainer
<code>FileNotFoundError:</code> <code>scaler.joblib</code>	Olvidaste entrenar	<code>python -m ml.anomaly.trainer --</code> <code>csv ml/data/synthetic.csv</code>

2.3 Layer 2 — Shannon entropy

Contexto

Señal complementaria barata. Ransomware cifra → bytes parecen aleatorios → entropía → ~8.0. Software legítimo escribe formatos estructurados → entropía 4-6.

`ml/entropy/shannon.py`



```

"""Shannon entropy de bloques de bytes."""

from __future__ import annotations
import math
from collections import Counter

def shannon_entropy(data: bytes) → float:
    """Entropy in bits per byte. Max = 8.0 (uniformly random)."""
    if not data:
        return 0.0
    counts = Counter(data)
    n = len(data)
    return -sum((c / n) * math.log2(c / n) for c in counts.values())

def classify_write(entropy: float) → str:
    if entropy ≥ 7.5: return "encrypted_or_random"
    if entropy ≥ 6.5: return "compressed"
    if entropy ≥ 4.5: return "structured_text_or_binary"
    return "low_entropy"

```

Tests completos



```
# ml/entropy/tests/test_shannon.py
import os
import pytest
from ml.entropy.shannon import shannon_entropy, classify_write

def test_empty():
    assert shannon_entropy(b'') == 0.0

def test_single_byte_value():
    assert shannon_entropy(b'\x00' * 1000) == pytest.approx(0.0)

def test_uniform_near_max():
    data = bytes(range(256)) * 4
    assert shannon_entropy(data) == pytest.approx(8.0, abs=0.01)

def test_random_bytes_close_to_8():
    assert shannon_entropy(os.urandom(8192)) > 7.9

def test_ascii_text_lower():
    text = b"the quick brown fox jumps over the lazy dog " * 200
    assert 4.0 < shannon_entropy(text) < 5.0

def test_classify_buckets():
    assert classify_write(7.9) == "encrypted_or_random"
    assert classify_write(6.9) == "compressed"
    assert classify_write(5.0) == "structured_text_or_binary"
    assert classify_write(2.0) == "low_entropy"
```

Verificación



```
pytest ml/entropy/tests/ -v
```

Esperado:

SALIDA ESPERADA

```
test_shannon.py::test_empty PASSED
test_shannon.py::test_single_byte_value PASSED
test_shannon.py::test_uniform_near_max PASSED
test_shannon.py::test_random_bytes_close_to_8 PASSED
test_shannon.py::test_ascii_text_lower PASSED
test_shannon.py::test_classify_buckets PASSED
===== 6 passed in 0.05s =====
```

2.4 Layer 4 — Cliente LLM dual (OpenAI + Llama)

Contexto

Interfaz `LLMClient` con dos implementaciones intercambiables vía `LLM_BACKEND`. El consumer no sabe cuál usa.

```
ml/llm_triage/client.py
```

The Python logo, consisting of three colored circles (red, yellow, green) arranged in a triangular pattern, with the word "PYTHON" in white capital letters below them.

```

"""Cliente LLM unificado: OpenAI primario, Llama local fallback."""

from __future__ import annotations
import os, time, json
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Literal, Optional
import httpx

@dataclass(frozen=True)
class LLMVerdict:
    label: Literal["malicious", "benign", "uncertain"]
    confidence: float
    reasoning: str
    backend: str
    latency_ms: int

class LLMClient(ABC):
    @abstractmethod
    async def classify(self, prompt: str) → LLMVerdict: ...

class OpenAIClient(LLMClient):
    def __init__(self, api_key: Optional[str] = None,
                 model: str = "gpt-4o-mini", timeout: float = 10.0):
        self._key = api_key or os.environ["OPENAI_API_KEY"]
        self._model = model
        self._http = httpx.AsyncClient(timeout=timeout)

    async def classify(self, prompt: str) → LLMVerdict:
        t0 = time.monotonic()
        r = await self._http.post(
            "https://api.openai.com/v1/chat/completions",
            headers={"Authorization": f"Bearer {self._key}"},
            json={
                "model": self._model,
                "messages": [
                    {"role": "system",
                     "content": "You are a SOC analyst. Respond with strict JSON: "
                                '{"label\":\"malicious|benign|uncertain\",'
                                '\"confidence\":0.0..1.0,\"reasoning\":\"...\"}"},
                    {"role": "user", "content": prompt},
                ]
            }
        )

```



```

        ],
        "response_format": {"type": "json_object"},
        "temperature": 0.0,
    },
)
r.raise_for_status()
parsed = json.loads(r.json()["choices"][0]["message"]["content"])
return LLMVerdict(
    label=parsed["label"], confidence=float(parsed["confidence"]),
    reasoning=parsed["reasoning"], backend="openai_gpt4o_mini",
    latency_ms=int((time.monotonic() - t0) * 1000),
)

class OllamaClient(LLMClient):
    def __init__(self, model: str = "llama3.1:8b-instruct-q4_K_M",
                 base_url: str = "http://localhost:11434",
                 timeout: float = 30.0):
        self._model = model
        self._url = base_url
        self._http = httpx.AsyncClient(timeout=timeout)

    async def classify(self, prompt: str) → LLMVerdict:
        t0 = time.monotonic()
        r = await self._http.post(
            f"{self._url}/api/chat",
            json={
                "model": self._model,
                "messages": [
                    {"role": "system",
                     "content": "Respond JSON only: "
                                "{\n\"label\":..., \"confidence\":..., \"reasoning\":...,\n"
                                "\"role\": \"user\", \"content\": prompt},\n"
                                "],\n"
                                "\"format\": \"json\", \"stream\": False,\n"
                                "\"options\": {\"temperature\": 0.0},\n"
                                "},\n"
                ],
                "format": "json", "stream": False,
                "options": {"temperature": 0.0},
            },
        )
        r.raise_for_status()
        parsed = json.loads(r.json()["message"]["content"])
        return LLMVerdict(
            label=parsed.get("label", "uncertain"),
            confidence=float(parsed.get("confidence", 0.5)),
            reasoning=parsed.get("reasoning", ""),
            backend="llama_local",
            latency_ms=int((time.monotonic() - t0) * 1000),
        )

```

```
)
```

```
def make_client() → LLMClient:
    backend = os.environ.get("LLM_BACKEND", "openai_gpt4o_mini")
    if backend == "openai_gpt4o_mini":
        return OpenAIClient()
    if backend == "llama_local":
        return OllamaClient()
    raise ValueError(f"unknown LLM_BACKEND: {backend}")
```

Verificación OpenAI

VERIFICACIÓN

```
export LLM_BACKEND=openai_gpt4o_mini
python - << 'PY'
import asyncio
from ml.llm_triage.client import make_client

async def main():
    c = make_client()
    v = await c.classify(
        "A process called encryptor.exe wrote 1500 files in 30 seconds with "
        "Shannon entropy 7.9. Is this malicious?"
    )
    print(v.label, v.confidence, v.backend, f"{v.latency_ms}ms")
asyncio.run(main())
PY
```

Esperado:

SALIDA ESPERADA

```
malicious 0.95 openai_gpt4o_mini 687ms
```

Verificación Llama



VERIFICACIÓN

```
export LLM_BACKEND=llama_local
python - << 'PY'
import asyncio
from ml.llm_triage.client import make_client

async def main():
    c = make_client()
    v = await c.classify("Same prompt as above.")
    print(v.label, v.confidence, v.backend, f"{v.latency_ms}ms")
asyncio.run(main())
PY
```

Esperado (con GPU):

SALIDA ESPERADA

```
malicious 0.92 llama_local 1200ms
```

Sin GPU el `latency_ms` puede ser ~4500.

Si algo falla

Síntoma	Causa	Fix
<code>401 Unauthorized</code> (OpenAI)	Key inválida	Revisa <code>.env</code> , vuelve a copiar del dashboard
<code>429 Rate limited</code>	Demasiadas llamadas	Solo durante burst tests; para demo es improbable
Llama responde no-JSON	Q4 quant a veces produce prefijos	Wrap con regex: <code>re.search(r"\{.*\}", content, re.DOTALL)</code>
<code>Connection refused 11434</code>	Ollama no corre	<code>sudo systemctl start ollama</code>

2.5 Layer 4 — RAG (BM25 + BGE-large + RRF)

Contexto

Antes de pasar la alerta cruda al LLM, recuperar 3-5 documentos relevantes (definiciones MITRE, runbook, variantes históricas) y inyectarlos como contexto. Reduce alucinación y mejora confidence.

Decisión (ADR-0001 v2): **sin cross-encoder reranker**. BM25 + BGE-large + RRF (Reciprocal Rank Fusion) es suficiente para el corpus pequeño que tenemos.

`ml/rag/index.py`



```

"""Indexa corpus markdown para BM25 + BGE."""

from __future__ import annotations
import json, pickle, re
from pathlib import Path
from dataclasses import dataclass, asdict

import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer

CORPUS_ROOT = Path("ml/rag/corpus")
INDEX_DIR = Path("ml/rag/_index")
EMBED_MODEL = "BAAI/bge-large-en-v1.5"

@dataclass
class Doc:
    id: str
    source: str
    title: str
    text: str

def _tokenize(text: str) → list[str]:
    return re.findall(r"[a-zA-Z0-9]+", text.lower())

def load_corpus() → list[Doc]:
    docs = []
    for md in CORPUS_ROOT.rglob("*.md"):
        text = md.read_text(encoding="utf-8")
        title = text.splitlines()[0].lstrip("# ").strip() if text else md.stem
        docs.append(Doc(id=str(md.relative_to(CORPUS_ROOT)),
                        source=md.parent.name, title=title, text=text))
    return docs

def build() → None:
    INDEX_DIR.mkdir(parents=True, exist_ok=True)
    docs = load_corpus()
    if not docs:
        raise RuntimeError(f"corpus vacío en {CORPUS_ROOT}")
    tokenized = [_tokenize(d.text) for d in docs]

```

```

bm25 = BM25Okapi(tokenized)
with (INDEX_DIR / "bm25.pkl").open("wb") as f:
    pickle.dump(bm25, f)
model = SentenceTransformer(EMBED_MODEL)
embeddings = model.encode([d.text for d in docs], normalize_embeddings=True)
np.save(INDEX_DIR / "embeddings.npy", embeddings)
with (INDEX_DIR / "docs.jsonl").open("w") as f:
    for d in docs:
        f.write(json.dumps(asdict(d)) + "\n")
print(f"indexed {len(docs)} docs (BM25 + BGE-{embeddings.shape[1]}d)")

if __name__ == "__main__":
    build()

```

ml/rag/retriever.py



```

"""Retriever híbrido BM25 + denso, fusión con Reciprocal Rank Fusion."""

from __future__ import annotations
import json, pickle
from typing import Iterable

import numpy as np
from sentence_transformers import SentenceTransformer

from ml.rag.index import INDEX_DIR, EMBED_MODEL, Doc, _tokenize

RRF_K = 60

def _rrf(rankings: Iterable[list[int]], k: int = RRF_K) → list[tuple[int, float]]:
    scores: dict[int, float] = {}
    for rank in rankings:
        for pos, doc_idx in enumerate(rank):
            scores[doc_idx] = scores.get(doc_idx, 0.0) + 1.0 / (k + pos + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)

class HybridRetriever:
    def __init__(self):
        with (INDEX_DIR / "bm25.pkl").open("rb") as f:
            self._bm25 = pickle.load(f)
        self._embeds = np.load(INDEX_DIR / "embeddings.npy")
        with (INDEX_DIR / "docs.jsonl").open() as f:
            self._docs = [Doc(**json.loads(line)) for line in f]
        self._model = SentenceTransformer(EMBED_MODEL)

    def retrieve(self, query: str, k: int = 5) → list[Doc]:
        bm25_scores = self._bm25.get_scores(_tokenize(query))
        bm25_rank = list(np.argsort(bm25_scores)[::-1])
        qv = self._model.encode([query], normalize_embeddings=True)[0]
        dense_scores = self._embeds @ qv
        dense_rank = list(np.argsort(dense_scores)[::-1])
        fused = _rrf([bm25_rank[:30], dense_rank[:30]])
        return [self._docs[idx] for idx, _ in fused[:k]]

```

Pasos manuales — poblar el corpus

1. Crea estructura:



BASH

```
mkdir -p ml/rag/corpus/mitre ml/rag/corpus/runbook ml/rag/corpus/variants
```

1. Para cada técnica MITRE relevante (T1486, T1490, T1083, T1190, T1498, T1499), crea un archivo `ml/rag/corpus/mitre/<técnica>.md` con: título, descripción tomada de attack.mitre.org, ejemplos comunes.
2. Para runbooks, crea `ml/rag/corpus/runbook/<nombre>.md` con instrucciones del equipo SOC.
3. Para variantes históricas, crea `ml/rag/corpus/variants/<familia>.md` (LockBit, Conti, REvil, etc.) con descripciones técnicas.

Comandos (corpus de smoke test)



BASH


```
mkdir -p ml/rag/corpus/mitre ml/rag/corpus/runbook
cat > ml/rag/corpus/mitre/T1486.md << 'EOF'
# T1486 – Data Encrypted for Impact
```

Adversaries may encrypt data on target systems or large numbers of systems in a network to interrupt availability. Ransomware families commonly observed: LockBit, Conti, REvil, Ryuk, BlackCat.

Indicators: high file-write entropy (~8.0), mass file rename to .locked extension, deletion of volume shadow copies via vssadmin.

EOF

```
cat > ml/rag/corpus/runbook/ransomware_response.md << 'EOF'
# Runbook – Ransomware response
```

If file-write entropy > 7.5 AND files_touched_per_min > 100, isolate host immediately. Do NOT wait for confirmation; encryption is destructive and fast.

Steps:

1. Trigger Wazuh active-response to disable host network.
2. Snapshot memory if forensics capability available.
3. Notify SOC manager via Telegram + Discord.

EOF

```
python -m ml.rag.index
```

Salida esperada

(Primera vez descarga BGE-large, puede tomar 1-2 min):

SALIDA ESPERADA

```
indexed 2 docs (BM25 + BGE-1024d)
```

Verificación


VERIFICACIÓN

```
python - << 'PY'
from ml.rag.retriever import HybridRetriever
r = HybridRetriever()
for d in r.retrieve("Process encrypting files in bulk", k=2):
    print(d.id, ">", d.title)
PY
```

Esperado:

SALIDA ESPERADA

```
mitre/T1486.md → T1486 – Data Encrypted for Impact
runbook/ransomware_response.md → Runbook – Ransomware response
```

Si algo falla

Síntoma	Causa	Fix
<code>corpus vacío</code>	No hay archivos <code>.md</code> en <code>corpus/</code>	Agrega al menos 1 archivo <code>.md</code> en <code>ml/rag/corpus/<algo>/</code>
Descarga BGE muy lenta	Conexión a HuggingFace bloqueada	<code>export HF_ENDPOINT=https://hf-mirror.com</code>
Memoria insuficiente con BGE-large	RAM limitada	Cambia a <code>BAAI/bge-base-en-v1.5</code> (~400 MB) en <code>EMBED_MODEL</code>

2.6 Layer 4 — Triage end-to-end

`ml/llm_triage/triage.py`

 PYTHON

```

"""Ensambla: evento → prompt con contexto RAG → LLM → LLMVerdict."""

from __future__ import annotations
import logging
from typing import Optional

from argos_contracts.incident import NormalizedEvent
from ml.llm_triage.client import LLMVerdict, make_client
from ml.rag.retriever import HybridRetriever

logger = logging.getLogger(__name__)

_RETRIEVER: Optional[HybridRetriever] = None
_CLIENT = None

def _retriever() → HybridRetriever:
    global _RETRIEVER
    if _RETRIEVER is None:
        _RETRIEVER = HybridRetriever()
    return _RETRIEVER

def _client():
    global _CLIENT
    if _CLIENT is None:
        _CLIENT = make_client()
    return _CLIENT

def _build_prompt(event: NormalizedEvent, ctx_docs) → str:
    ctx = "\n\n".join(f"### {d.title}\n{d.text[:600]}" for d in ctx_docs)
    return (
        f"## Event\n"
        f"Host: {event.host}\n"
        f"MITRE technique: {event.mitre_technique}\n"
        f"Severity: {event.severity}\n"
        f"Layers fired: {event.num_layers_fired}\n"
        f"Confidence (stat): {event.confidence_score}\n"
        f"\n## Context (top RAG docs)\n{ctx}\n"
        f"\n## Task\nClassify the event."
    )

```

```

async def classify(event: NormalizedEvent) → LLMVerdict:
    docs = _retriever().retrieve(f"{event.mitre_technique} {event.host}", k=4)
    prompt = _build_prompt(event, docs)
    return await _client().classify(prompt)

```

Y exporta:

 PYTHON

```

# ml/llm_triage/__init__.py
from ml.llm_triage.triage import classify
__all__ = ["classify"]

```

Verificación

 VERIFICACIÓN

```

python - << 'PY'
import asyncio
from argos_contracts.incident import NormalizedEvent
from argos_contracts.enums import Severity
from ml.llm_triage import classify

evt = NormalizedEvent(
    event_id="evt-llm-001", severity=Severity.MEDIUM,
    mitre_technique="T1486", num_layers_fired=2, confidence_score=0.71,
    host="WIN-VICTIM-01", layer_origin="sigma",
)
v = asyncio.run(classify(evt))
print(v.label, v.confidence, v.backend, f"{v.latency_ms}ms")
PY

```

Esperado:

SALIDA ESPERADA

malicious 0.93 openai_gpt4o_mini 750ms

Checklist Fase 2

#	Check	OK
1	Synthetic dataset (5201 filas)	☐
2	3 modelos <code>.joblib</code> entrenados	☐
3	TP $\geq$ 85 %, FP $\leq$ 5 %	☐
4	Shannon entropy: 6 tests passed	☐
5	OpenAI client devuelve verdict	☐
6	Llama client devuelve verdict	☐
7	RAG indexa corpus y recupera por similaridad	☐
8	<code>classify(event)</code> end-to-end < 2 s con OpenAI	☐
9	<code>pytest ml/ -q</code> $\rightarrow \geq 20$ passed	☐

Fase 3 — Integración real

3.1 Consumer del stream `events:raw_wazuh` → emite `events:normalized`

Contexto

Te suscribes al stream que P3 emite desde Wazuh (`events:raw_wazuh`). Para cada evento computas features, corres Layer 2, opcionalmente Layer 4, y emites `NormalizedEvent` al stream que P1 consume.

`m1/consumer.py`



```

"""Pipeline ML: raw Wazuh event → features → Layer 2 + (opcional) Layer 4 → normalized

from __future__ import annotations
import asyncio, json, logging, os, uuid
import redis.asyncio as redis

from argos_contracts.enums import Severity
from argos_contracts.incident import NormalizedEvent
from ml.anomaly.scorer import AnomalyScorer

logger = logging.getLogger(__name__)

IN_STREAM = "events:raw_wazuh"
OUT_STREAM = "events:normalized"
GROUP = "ml-pipeline"
CONSUMER = os.environ.get("ML_CONSUMER_NAME", "ml-1")

_scorer = AnomalyScorer.load()

def _features_from_wazuh(evt: dict) → dict:
    return {
        "syscalls_per_min": evt.get("syscalls_per_min", 0),
        "files_touched_per_min": evt.get("files_touched_per_min", 0),
        "entropy_of_written_bytes": evt.get("entropy_of_written_bytes", 0.0),
        "network_kbps": evt.get("network_kbps", 0),
        "command_line_len": len(evt.get("command_line", "")),
    }

async def _process(r: redis.Redis, raw: dict) → None:
    evt = json.loads(raw["data"])
    feats = _features_from_wazuh(evt)
    score = _scorer.score(feats)
    if not score.is_anomaly:
        return
    norm = NormalizedEvent(
        event_id=f"evt-{uuid.uuid4().hex[:12]}",
        severity=Severity.MEDIUM if score.confidence < 0.85 else Severity.HIGH,
        mitre_technique=evt.get("mitre_technique", "Unknown"),
        num_layers_fired=1,
        confidence_score=score.confidence,
        host=evt.get("host", "unknown"),
        layer_origin="ml",
    )

```

```

    )
    await r.xadd(OUT_STREAM, {"data": norm.model_dump_json()})
    logger.info("ml emit %s host=%s conf=%.2f",
                norm.event_id, norm.host, norm.confidence_score)

async def run() → None:
    r = redis.from_url(os.environ["REDIS_URL"], decode_responses=True)
    try:
        await r.xgroup_create(IN_STREAM, GROUP, id="0", mkstream=True)
    except redis.ResponseError as e:
        if "BUSYGROUP" not in str(e):
            raise
    while True:
        resp = await r.xreadgroup(GROUP, CONSUMER, {IN_STREAM: ">"}, count=10, block=1)
        for _, entries in resp or []:
            for entry_id, fields in entries:
                try:
                    await _process(r, fields)
                    await r.xack(IN_STREAM, GROUP, entry_id)
                except Exception: # noqa: BLE001
                    logger.exception("failed to process %s", entry_id)

if __name__ == "__main__":
    asyncio.run(run())

```

Comandos

 BASH

```

# Terminal A
python -m ml.consumer

# Terminal B (inyectar evento Wazuh simulado)
redis-cli XADD events:raw_wazuh '*' data \
    '{"host":"WIN-VICTIM-01","mitre_technique":"T1486","syscalls_per_min":5500,"files'

```


Salida esperada (terminal A)

SALIDA ESPERADA

```
INFO:ml.consumer:ml emit evt-abc123 host=WIN-VICTIM-01 conf=0.93
```

Verificación

VERIFICACIÓN

```
redis-cli XLEN events:normalized
redis-cli XREVRANGE events:normalized + - COUNT 1
```

Esperado (XLEN ≥ 1, último elemento es JSON con `event_id`, `confidence_score`, `host`):

SALIDA ESPERADA

```
1
1) 1) "1234-0"
   2) 1) "data"
      2) "{\"event_id\":\"evt-abc123\",\"severity\":\"HIGH\", ...}"
```

Si algo falla

Síntoma	Causa	Fix
Consumer no emite nada	Evento no pasa <code>is_anomaly</code>	Inyecta evento con <code>entropy=7.9</code> y <code>files_touched=1000</code>
<code>joblib.load</code> falla	Modelos no entrenados	<code>python -m ml.anomaly.trainer --csv ml/data/synthetic.csv</code>
<code>BUSYGROUP</code> warning	Grupo ya existe	Ignorar — el código lo maneja

3.2 Smoke end-to-end con P1

Coordina con P1 una corrida conjunta: él levanta su consumer SOAR, tú tu consumer ML, P3 (o tú con un comando manual) inyecta a `events:raw_wazuh`. Verifica que `incident:*` aparece en Redis y la notif llega a Telegram con `layer_origin=ml` en los logs.

Verificación

VERIFICACIÓN

```
redis-cli KEYS 'incident:*' | head -5
redis-cli GET $(redis-cli KEYS 'incident:*' | tail -1) | jq '.llm_verdict.label, .l'
```

Esperado (para un T2):

SALIDA ESPERADA

```
incident:inc-abc123def456
"malicious"
"openai_gpt4o_mini"
```

Checklist Fase 3

#	Check	OK
1	ML consumer corre y procesa stream	☐
2	Emisión solo cuando <code>is_anomaly</code>	☐
3	Integración con P1 verificada end-to-end	☐
4	<code>llm_verdict</code> poblado para T2	☐
5	Audit log (P1) refleja <code>layer_origin=ml</code>	☐

Fase 4 — Rehearsal y polish

4.1 Rehearsal de carga

Comandos

 BASH

```
for i in $(seq 1 50); do
  redis-cli XADD events:raw_wazuh '*' data \
    "{\"host\":\"WIN-VICTIM-01\",\"mitre_technique\":\"T1486\",\"syscalls_per_min\"
done
sleep 5
redis-cli XLEN events:normalized
```

Verificación

 VERIFICACIÓN

```
test $(redis-cli XLEN events:normalized) -ge 30 && echo "throughput OK"
ps -o rss= -p $(pgrep -f 'ml.consumer') | awk '{print "ML RSS: "$1/1024" MB"}'
```

Esperado:

SALIDA ESPERADA

```
throughput OK
ML RSS: 480 MB
```

(Memoria estable entre antes y después del burst = no leak.)

4.2 Failover OpenAI → Llama

Pasos manuales

1. Simular OpenAI caído invalidando la key:



BASH

```
export OPENAI_API_KEY=sk-invalid-test
```

2. Inyectar un evento T2 al stream. El consumer P1 debería loguear `llm classify failed; continuing without enrichment` y NO crashear.
3. Switch a Llama:



BASH

```
export LLM_BACKEND=llama_local
```

4. Reinicia ML consumer. Los siguientes eventos enriquecen con Llama.

Verificación



VERIFICACIÓN

```
# Después del switch:
redis-cli XADD events:raw_wazuh '*' data '{"host":"WIN-VICTIM-01","mitre_technique"
sleep 5
redis-cli GET $(redis-cli KEYS 'incident:*' | tail -1) | jq '.llm_verdict.backend'
```

Esperado:

SALIDA ESPERADA

"llama_local"

4.3 Pre-cache de embeddings (T-1h del demo)

Comandos



BASH

```
python - << 'PY'
from ml.rag.retriever import HybridRetriever
_ = HybridRetriever()
print("RAG retriever ready")
PY
```

Salida esperada (primera vez)

SALIDA ESPERADA

RAG retriever ready

(Tras 5-15 s.)

Mantén el proceso vivo durante el demo o expone como servicio con `uvicorn` .

Checklist Fase 4

#	Check	OK
1	Burst test 50 eventos en ≤ 3 s	☐
2	Failover OpenAI $\rightarrow$ Llama probado	☐
3	RAG pre-warmed antes del demo	☐
4	TP/FP registrados en <code>docs/LESSONS_LEARNED.md</code>	☐
5	Rehearsal con P1 y P3 cerrado	☐

Apéndice A — Troubleshooting

#	Síntoma	Diagnóstico	Fix
A. 1	<code>sentence-transformers</code> descarga falla	HuggingFace bloqueado	<code>export HF_ENDPOINT=https://hf-mirror.com</code>
A. 2	OpenAI <code>429</code>	Rate limit	Bajar concurrencia; para demo no aplica
A. 3	Llama responde no-JSON	Q4 prefijos	Regex extractor <code>re.search(r"\{.*\}", content, re.DOTALL)</code>
A. 4	IsoForest predice todo como anomalía	Scaler no cargado en inferencia	Carga <code>joblib.load(scaler.joblib)</code> y aplica a TODOS los features
A. 5	BM25 scores cero	Query tokenizada distinto	Usa <code>_tokenize</code> del módulo <code>index</code>
A. 6	Ollama "model not loaded"	Modelo descargado pero no cargado	<code>ollama run llama3.1:8b-instruct-q4_K_M ""</code> para forzar carga
A. 7	Memoria insuficiente con BGE-large	RAM limitada	Cambia a BGE-base (~400 MB) en <code>EMBED_MODEL</code>

Apéndice B — Comandos de emergencia



```

# Reentrenar modelos en frío (~5 min)
python -m ml.data.synthetic_generator --benign 5000 --attack 200
python -m ml.anomaly.trainer --csv ml/data/synthetic.csv

# Reconstruir índice RAG (1-2 min)
python -m ml.rag.index

# Forzar Llama (sin tocar P1)
export LLM_BACKEND=llama_local
pkill -f 'ml.consumer' && nohup python -m ml.consumer > /tmp/ml.log 2>&1 &

# Liberar memoria si Ollama acapara
ollama stop llama3.1:8b-instruct-q4_K_M

# Resetear consumer group ML
redis-cli XGROUP DESTROY events:raw_wazuh ml-pipeline
redis-cli XGROUP CREATE events:raw_wazuh ml-pipeline 0 MKSTREAM

```

Apéndice C — Referencias cruzadas

Cuando estés en...	Lee
ml/anomaly/	SAD §6.3, ADR-0001 v2
ml/llm_triage/	SAD §6.5, ADR-0001 v2
ml/rag/	SAD §6.5.2, docs/CONTEXT.md
Calibración Q5 (post-demo)	docs/CONTEXT.md Q3

Change log

Versión	Fecha	Cambio
3.0	2026-05-24	Reestructurado: Contexto → Pasos manuales → Comandos → Salida → Verificación → Si algo falla. Bloques bash listos para copy buttons. Sin referencias temporales. Renombrado de <code>sprint-week-1-p2-sebastian.md</code> .